Guia do Grupo - Stack Mais Facil e Inicio das Tasks

Objetivo

Este guia foi escrito para o grupo com foco em rapidez de aprendizagem, baixo atrito tecnico e inicio imediato das tasks ja criadas. A ideia e simples: comecar com a stack mais leve para o MVP, quebrar o trabalho em pecas pequenas e evitar que um aluno fique travado esperando outro.

Stack Recomendada para o Grupo

A stack mais facil para este momento do projeto e:

- `Vite`
- `React`
- `JavaScript`
- `React Router`
- `Supabase`
- `CSS Modules` com variaveis CSS globais

Por Que Esta Stack e a Mais Facil

1. Menos camadas para aprender

- `Vite` sobe rapido e tem configuracao simples
- `React` resolve interface e componentes com bastante material de apoio
- `JavaScript` evita a curva extra do TypeScript neste primeiro momento
- `React Router` resolve navegacao sem trazer peso desnecessario
- `Supabase` permite usar banco e consultas sem construir um backend completo do zero

2. Uma base so para Web e Mobile

Neste MVP, `mobile` significa `web mobile-first`, nao app nativo separado. Isso reduz custo, tempo e confusao.

3. Menos dependencia entre colegas

Com `mock local + service simples`, cada aluno consegue construir sua parte sem esperar outra task ser finalizada.

O Que Nao Vamos Fazer Agora

Para manter a entrada facil, o grupo nao deve comecar com:

- `Next.js`
- `Expo`
- `TypeScript`
- `microservicos`
- `SSR`
- `autenticacao complexa`
- `notificacoes push`
- `PWA avancado`

Esses temas podem voltar depois, mas nao devem bloquear o inicio do MVP.

Instalacao Base

Pre-requisitos

- `Node.js` LTS instalado
- `npm` funcionando
- conta no GitHub
- acesso ao projeto Supabase do SIBiSC

Comandos iniciais

```
npm create vite@latest sibisc-app -- --template react
cd sibisc-app
npm install
npm install react-router-dom @supabase/supabase-js
npm run dev
```

Estrutura Minima Recomendada

```
src/
  assets/
  components/
  lib/
  mocks/
  pages/
  routes/
  services/
  styles/
  utils/
```

Funcao de cada pasta

- `components/` : pecas reutilizaveis como cards, cabecalho e estados de tela
- `pages/` : telas principais como noticias, eventos e acervo
- `mocks/` : dados locais para trabalhar sem depender de backend real
- `services/` : funcoes que buscam dados, primeiro em mock e depois no Supabase
- `lib/` : configuracoes como cliente do Supabase
- `styles/` : variaveis globais e estilos compartilhados
- `routes/` : definicao das rotas do app
- `utils/` : funcoes pequenas de apoio

Variaveis de Ambiente

Criar `.env` com:

```
VITE_SUPABASE_URL=https://vuhfazknlyqnphriilkw.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_UklsEfW9sJJP5ScGJItGVw_OhJRQ2Kf
```

Regra de Seguranca

Nunca colocar no repositorio:

- senha do banco
- `service role key`
- credenciais privadas de qualquer servico

Regra Principal Para Nao Gerar Dependencias Entre Tasks

Toda task deve comecar em modo `independente`.

Como fazer isso na pratica

1. criar a tela ou componente com `mock local`
2. usar um `service` simples para ler esses mocks
3. validar layout, navegacao e comportamento localmente
4. documentar a entrega
5. so depois o TL ou a dupla de integracao troca a fonte mock pela fonte real

Exemplo de Estrategia Sem Dependencia

Errado

- aluno A espera o backend ficar pronto para criar a tela
- aluno B espera a tela ficar pronta para criar o detalhe
- aluno C espera os dois para testar

Certo

- aluno A cria tela de noticias com mock
- aluno B cria detalhe de noticia com mock
- aluno C cria card de noticia reutilizavel com mock
- depois o TL conecta os tres com a fonte de dados real

Padrao de Service Simples

Cada dominio deve ter um service inicial bem pequeno.

Exemplo

```
export async function getNews() {
  return newsMock;
}

export async function getNewsById(id) {
  return newsMock.find((item) => item.id === id);
}
```

Depois, quando a integracao real entrar, o arquivo muda por dentro, mas a tela continua chamando a mesma funcao.

Ordem de Aprendizagem Recomendada

Nivel 1 - Muito Facil

Ideal para quem esta pegando ritmo no projeto.

1. **`T-BASE-003` - cabecalho e menu principal**
   - dificuldade: `muito facil`
   - dependencia: `nenhuma`
   - entrega: cabecalho com links para `Noticias`, `Eventos` e `Acervo`
   - aprendizado: componentes, layout e navegacao visual
2. **`T-BASE-003` - estados vazios iniciais**
   - dificuldade: `muito facil`
   - dependencia: `nenhuma`
   - entrega: componentes visuais para `loading`, `erro` e `sem resultados`
   - aprendizado: reaproveitamento e UX basica
3. **`T-NOT-001` - card de noticia**
   - dificuldade: `muito facil`
   - dependencia: `nenhuma`
   - entrega: card com titulo, data, resumo e botao de abrir
   - aprendizado: props, composicao e responsividade
4. **`T-EVT-001` - card de evento**
   - dificuldade: `muito facil`
   - dependencia: `nenhuma`
   - entrega: card com titulo, data, local e chamada visual
   - aprendizado: componente reutilizavel
5. **`T-ACV-002` - componente de disponibilidade**
   - dificuldade: `muito facil`
   - dependencia: `nenhuma`
   - entrega: lista simples com unidade e status `disponivel` ou `indisponivel`
   - aprendizado: renderizacao condicional
6. **`T-BASE-004` - documentacao da propria entrega**
   - dificuldade: `muito facil`
   - dependencia: `nenhuma`
   - entrega: registro simples do que foi feito e como validar
   - aprendizado: rastreabilidade

Nivel 2 - Facil

Ideal para quem ja conseguiu fazer um componente e quer montar uma tela inteira.

7. **`T-NOT-001` - pagina de noticias com mock**
   - dificuldade: `facil`
   - dependencia: `nenhuma`, desde que use mock local
   - entrega: listagem de noticias com cards e estados de tela
   - aprendizado: pagina, lista e dados mockados
8. **`T-NOT-002` - detalhe de noticia com mock**
   - dificuldade: `facil`

- dependencia: `nenhuma`, desde que use mock local
- entrega: tela de detalhe com titulo, data, conteudo e origem
- aprendizado: organizacao de conteudo longo

9. `T-EVT-001` - pagina de eventos com mock

- dificuldade: `facil`
- dependencia: `nenhuma`, desde que use mock local
- entrega: lista de eventos em ordem por data
- aprendizado: listas e informacao util ao usuario

10. `T-EVT-003` - detalhe de evento com mock

- dificuldade: `facil`
- dependencia: `nenhuma`, desde que use mock local
- entrega: tela com descricao, horario, local e publico-alvo
- aprendizado: detalhe de fluxo

11. `T-ACV-001` - campo de busca com mock

- dificuldade: `facil`
- dependencia: `nenhuma`, desde que use mock local
- entrega: busca por texto e lista de resultados local
- aprendizado: estado, input e filtro

12. `T-EVT-002` - calendario simplificado com mock

- dificuldade: `facil`
- dependencia: `nenhuma`, desde que use mock local
- entrega: agrupamento de eventos por dia
- aprendizado: manipulacao de dados e apresentacao

Nivel 3 - Facil para Medio

Ideal para quem ja entende componente e tela, e agora quer trabalhar comportamento.

13. `T-NOT-003` - navegacao entre lista e detalhe

- dificuldade: `facil para medio`
- dependencia: `nenhuma`, se lista e detalhe usarem mocks locais
- entrega: clique no card levando ao detalhe e botao de voltar
- aprendizado: `React Router`

14. `T-EVT-004` - acao de agenda com fallback web

- dificuldade: `facil para medio`
- dependencia: `nenhuma`, pode ser feito com dados mockados
- entrega: botao que monta link de agenda ou exportacao simples
- aprendizado: integracao leve com recurso externo

15. `T-ACV-004` - detalhe do livro com disponibilidade e destaque

- dificuldade: `facil para medio`
- dependencia: `nenhuma`, desde que use mock local
- entrega: detalhe do livro com unidades e uma recomendacao visual
- aprendizado: combinacao de dados na mesma tela

16. `T-OPS-002` - exibir informacoes publicas da unidade

- dificuldade: `facil para medio`

- dependencia: `nenhuma`, se usar mock local
- entrega: bloco reutilizavel com nome, endereco, horario e contato da unidade
- aprendizado: reaproveitamento entre modulos

17. `T-ACV-003` - fallback manual de localizacao

- dificuldade: `facil para medio`
- dependencia: `nenhuma`
- entrega: seletor manual de unidade ou bairro quando localizacao nao existir
- aprendizado: UX de contingencia

Nivel 4 - Medio

Ideal para quem ja esta confortavel com telas e quer integrar dados reais.

18. `T-BASE-002` - conectar Supabase ao app

- dificuldade: `medio`
- dependencia: `nenhuma`, porque e uma task de base
- entrega: cliente Supabase configurado em `src/lib/`
- aprendizado: ambiente e integracao externa

19. `T-NOT-001` - trocar mock por leitura real de noticias

- dificuldade: `medio`
- dependencia: `nenhuma` para quem pegar a responsabilidade de integracao
- entrega: service de noticias lendo do Supabase
- aprendizado: consulta real e tratamento de erro

20. `T-EVT-001` - trocar mock por leitura real de eventos

- dificuldade: `medio`
- dependencia: `nenhuma` para quem pegar a responsabilidade de integracao
- entrega: service de eventos lendo do Supabase
- aprendizado: integracao controlada

21. `T-ACV-001` - trocar mock por busca real de livros

- dificuldade: `medio`
- dependencia: `nenhuma` para quem pegar a responsabilidade de integracao
- entrega: busca ligada a dados reais
- aprendizado: query e resposta de lista

22. `T-ACV-002` - trocar mock por disponibilidade real

- dificuldade: `medio`
- dependencia: `nenhuma` para quem pegar a responsabilidade de integracao
- entrega: service de disponibilidade real por unidade
- aprendizado: composicao de consulta

Nivel 5 - Medio para Avancado

Ideal para poucos alunos ou para o TL acompanhar mais de perto.

23. `T-ACV-003` - geolocalizacao real do navegador

- dificuldade: `medio para avancado`
- dependencia: `nenhuma`, se a tela ja aceitar fallback
- entrega: obter coordenadas e calcular unidade sugerida

- aprendizado: API do navegador e fluxo com permissao

24. `T-QA-001` a `T-QA-004`

- dificuldade: `medio para avancado`
- dependencia: `nenhuma`, porque QA pode revisar mesmo em mock
- entrega: evidencias, checagem de compatibilidade e feedback formal
- aprendizado: qualidade e validacao tecnica

Como Distribuir as Tasks no Grupo

Alunos iniciando agora

Podem pegar direto do `Nivel 1` e `Nivel 2`.

Alunos que ja entregaram uma task simples

Podem pegar do `Nivel 3`.

Alunos com mais seguranca ou com apoio do TL

Podem pegar do `Nivel 4` e `Nivel 5`.

Regra Para Nao Travar o Grupo

Se uma task parecer depender de outra, aplicar esta regra:

- criar mock local
- congelar o contrato de entrada e saida
- implementar a tela ou componente com esse contrato
- deixar a troca para dado real para uma task de integracao separada

Contratos Minimos Recomendados

Noticia

```
{
  id: 'not-001',
  title: 'Titulo',
  summary: 'Resumo',
  content: 'Conteudo',
  date: '2026-03-16',
  sourceUrl: 'https://...'
}
```

Evento

```
{
  id: 'evt-001',
  title: 'Oficina',
  description: 'Descricao',
  date: '2026-03-20',
```

```
  time: '14:00',
  location: 'Biblioteca X'
}
```

Livro

```
{
  id: 'liv-001',
  title: 'Livro',
  author: 'Autor',
  units: [
    { name: 'Biblioteca A', available: true }
  ]
}
```

Definition of Done para os Alunos

Uma task so esta pronta quando:

- funciona localmente
- esta responsiva em tela pequena
- usa nomes claros em arquivo e componente
- nao quebrou navegacao de outras partes
- tem evidencias simples de validacao
- teve documentacao minima atualizada

Sequencia Mais Segura Para Esta Semana

1. subir o app com `Vite + React`
2. criar cabecalho, menu e estados visuais base
3. criar cards de noticia, evento e disponibilidade
4. montar paginas com mock local
5. montar detalhes com mock local
6. ligar rotas
7. revisar mobile
8. so depois conectar Supabase

Resumo Para Mandar no Grupo

Se o grupo quiser uma frase curta para alinhar todo mundo:

> *Vamos comecar com `Vite + React + JavaScript + React Router + Supabase`, usando `mock local` primeiro para que cada pessoa consiga trabalhar sem depender da task de outro colega. O foco inicial e fazer telas e fluxos pequenos, responsivos e bem documentados. A integracao real entra depois, sem refazer a interface.*

Documentos Que o Grupo Deve Ler Junto com Este Guia

- `../management/plano_mvp_jira.md`
- `../management/sprints/README.md`
- `como_rodar_o_projeto.md`
- `../governance/feedback_por_task_e_subtask.md`